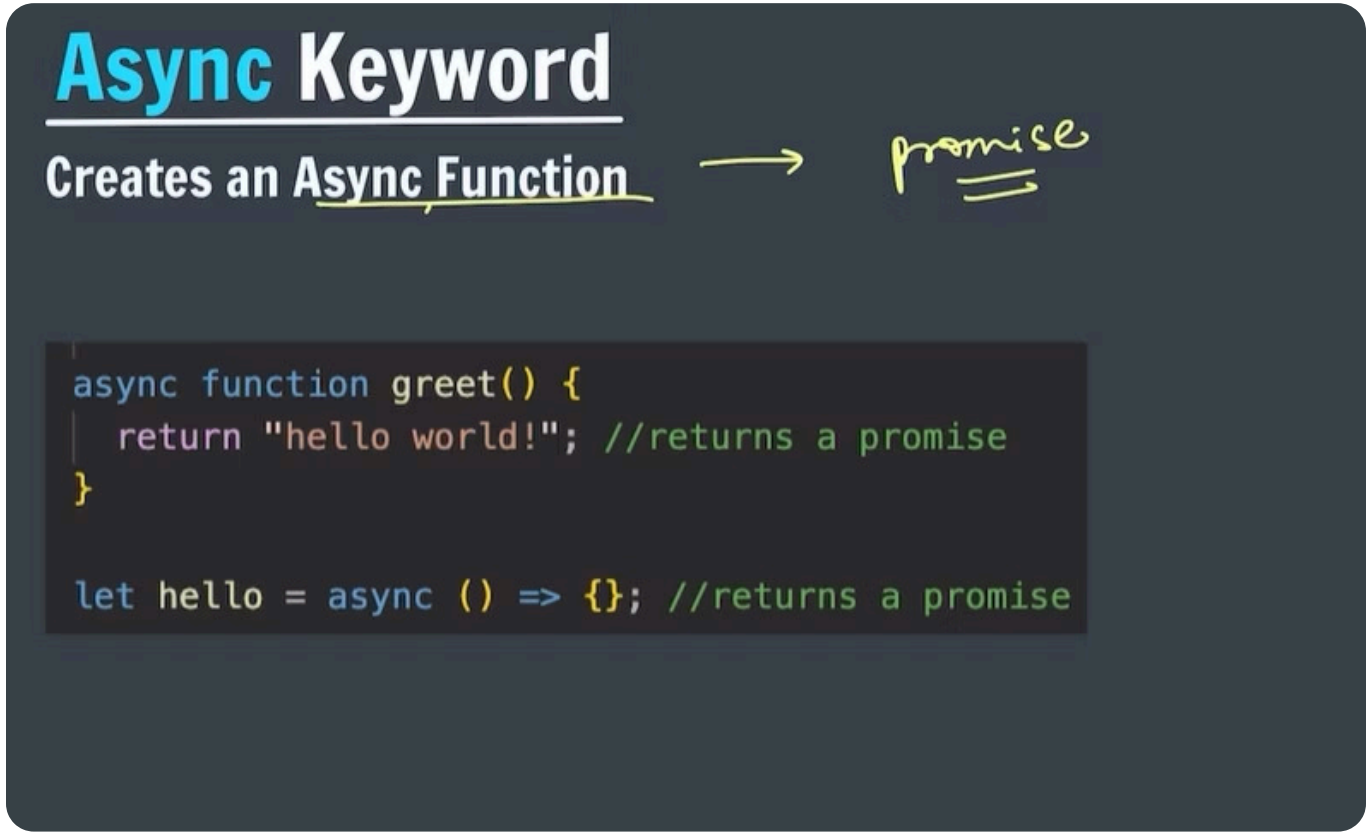
The async Keyword in JavaScript

A deep dive into async functions, Promises, and non-blocking execution — built for developers who want to write cleaner, more efficient asynchronous code.

What Does the async Keyword Do?



Creates an Async Function

Placing the `async` keyword before any function declaration or expression instantly transforms it into an **async function** — which always returns a **Promise**, regardless of what's inside.

- Regular functions return values directly
- Async functions wrap return values in a resolved Promise
- Works with both function declarations and arrow functions

Every async Function Returns a Promise

This is the fundamental contract: no matter what value you `return` inside an async function, the caller always receives a **Promise**. If you return `"hello world!"`, the caller gets `Promise<"hello world!">`.

Two Ways to Write an async Function

Function Declaration

Use `async` before the `function` keyword. Classic and explicit — great for named, reusable logic.

```
async function greet() {
  return "hello world!";
  // returns a Promise
}
```

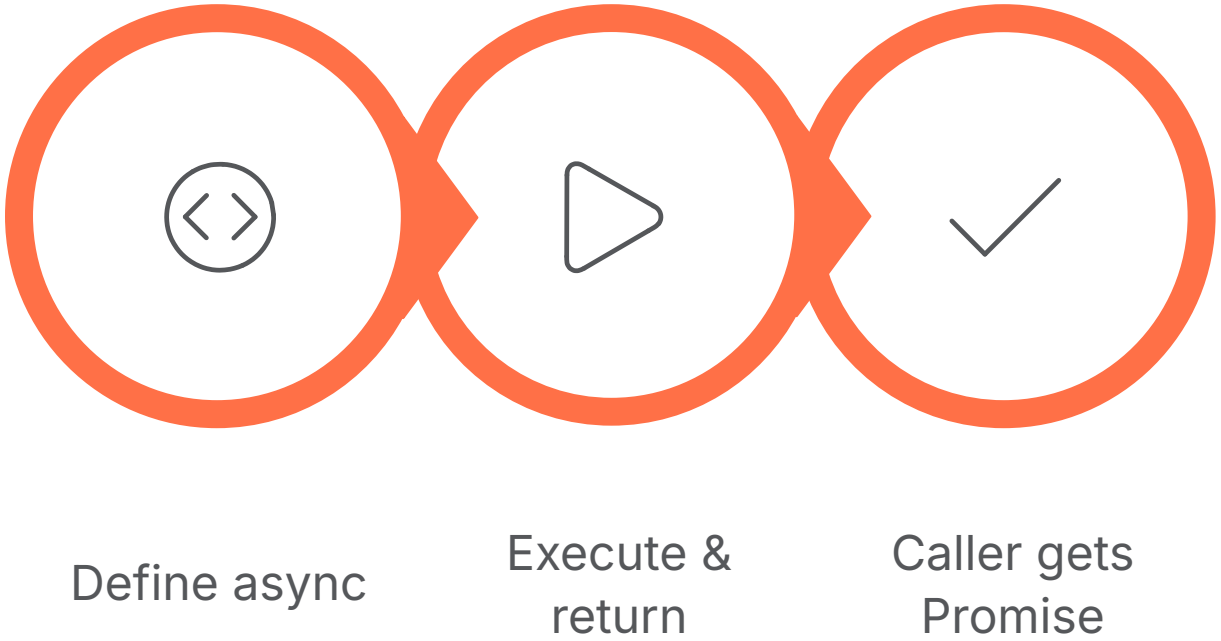
Arrow Function

Prefix the arrow function with `async`. Concise and ideal for callbacks or inline usage.

```
let hello = async () => {};
// also returns a Promise
```

Both forms are functionally equivalent in terms of Promise behavior — the choice is purely stylistic.

Under the Hood: What Gets Returned



Even a plain `return "hello"` inside an async function is automatically wrapped: the engine calls `Promise.resolve("hello")` on your behalf, keeping the interface consistent and chainable.

Error state

```
async function greet() {
  abc.abc();
  return "hello!";
}
```

```
async function greet() {
  throw "some random error"
  return "hello!";
}
```

```
greet()
  .then((result) => {
    console.log("promise was resolved");
    console.log("result was : ", result);
  })
  .catch((err) => {
    console.log("promise was rejected with err : ", err);
  });
```

Pairing async with await

```
function getNum() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let num = Math.floor(Math.random() * 10) + 1;
      console.log(num);
      resolve();
    }, 1000);
  });
}

async function demo() {
  await getNum();
  await getNum();
  await getNum();
  await getNum();
  getNum();
}
```

Await Keyword

pauses the execution of its surrounding async function until the promise is settled (resolved or rejected)

```
async function show() {
  await colorChange("violet", 1000);
  await colorChange("indigo", 1000);
  await colorChange("green", 1000);
  await colorChange("yellow", 1000);
  await colorChange("orange", 1000);

  return "done";
}
```

Lets Handle Rejection Case

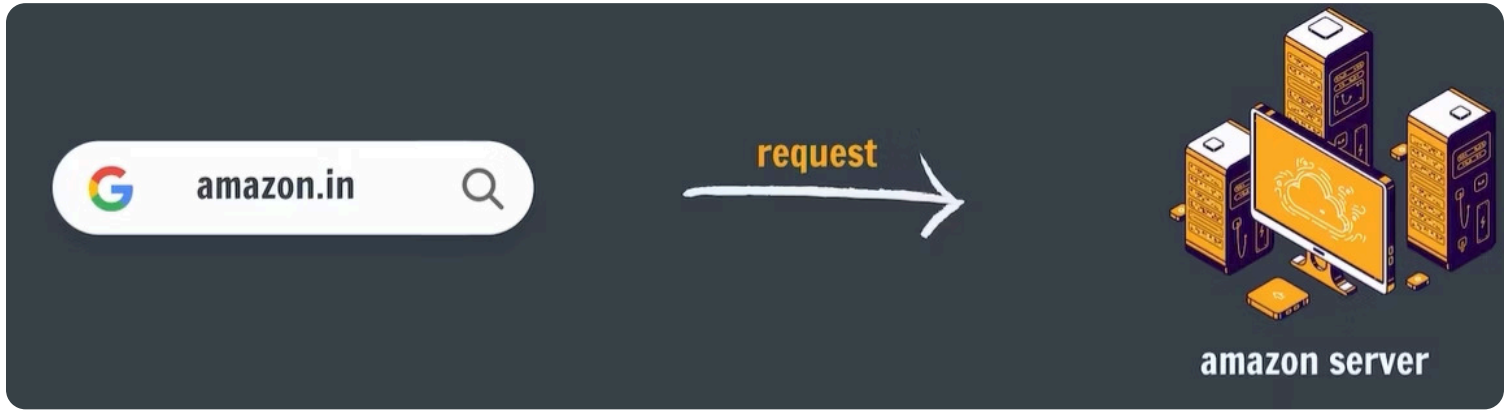
```
function changeColor(color, delay) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let num = Math.floor(Math.random() * 5) + 1;
      if (num > 3) {
        reject("promise rejected");
      }

      h1.style.color = color;
      console.log(`color changed to ${color}!`);
      resolve("color changed!");
    }, delay);
  });
}
```

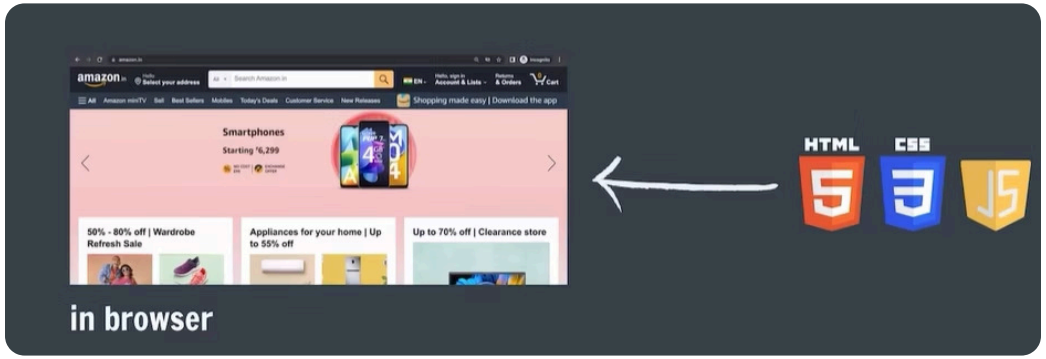
```
async function demo() {
  await changeColor("red", 1000);
  await changeColor("orange", 1000);
  await changeColor("green", 1000);
  await changeColor("blue", 1000);

  let a = 5;
  console.log(a);
  console.log("new number = ", a + 3);
}
```

```
try {
  await changeColor("red", 1000);
  await changeColor("orange", 1000);
  await changeColor("green", 1000);
  await changeColor("blue", 1000);
} catch (err) {
  console.log("error caught");
  console.log(err);
}
```

What is an API?

Application Programming Interface





What is JSON?

JavaScript Object Notation

JSON Example

```
1  {
2    "string": "Hi",
3    "number": 2.5,
4    "boolean": true,
5    "null": null,
6    "object": { "name": "Kyle", "age": 24 },
7    "array": ["Hello", 5, false, null, { "key": "value", "number": 6 }],
8    "arrayOfObjects": [
9      { "name": "Jerry", "age": 28 },
10     { "name": "Sally", "age": 26 }
11   ]
12 }
```

www.json.org

JSON

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Editio...

JSON → String

Accessing Data from JSON

- JSON.parse(data) Method

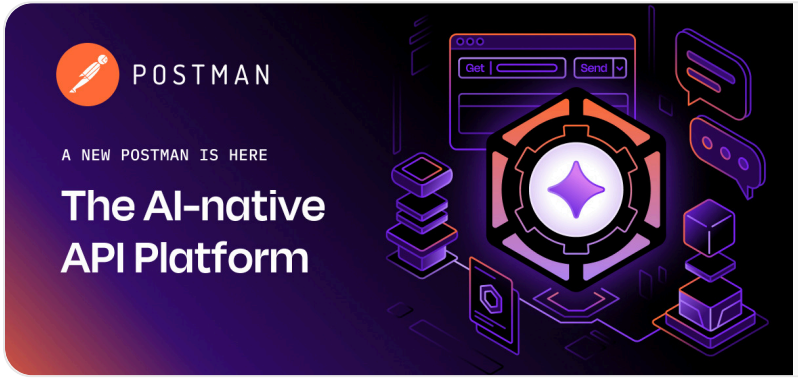
  To parse a string data into a JS object

- JSON.stringify(json) Method

  To parse a JS object data into JSON

```
let jsonRes =
  '{"fact":"Approximately 1/3 of cat owners think their pets are able to read their minds.","length":78}';

let validRes = JSON.parse(jsonRes);
console.log(validRes.fact);
```

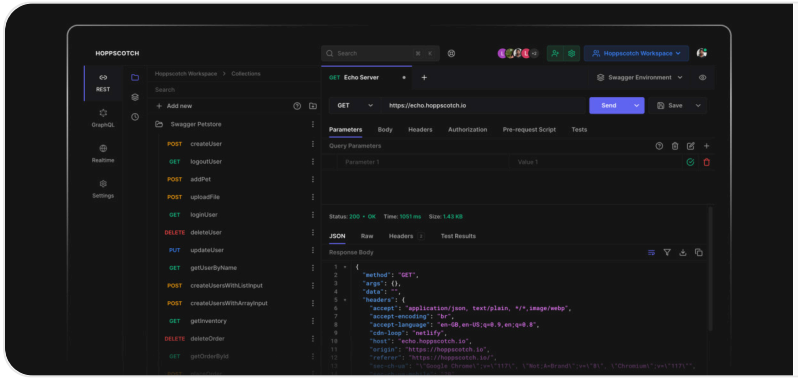
Testing API Requests



getpostman

Postman: The World's Leading API Platform | Sign Up for Free

Accelerate API development with Postman's all-in-one platform. Streamline collaboration and simplify the API lifecycle for faster, better results. Learn more.



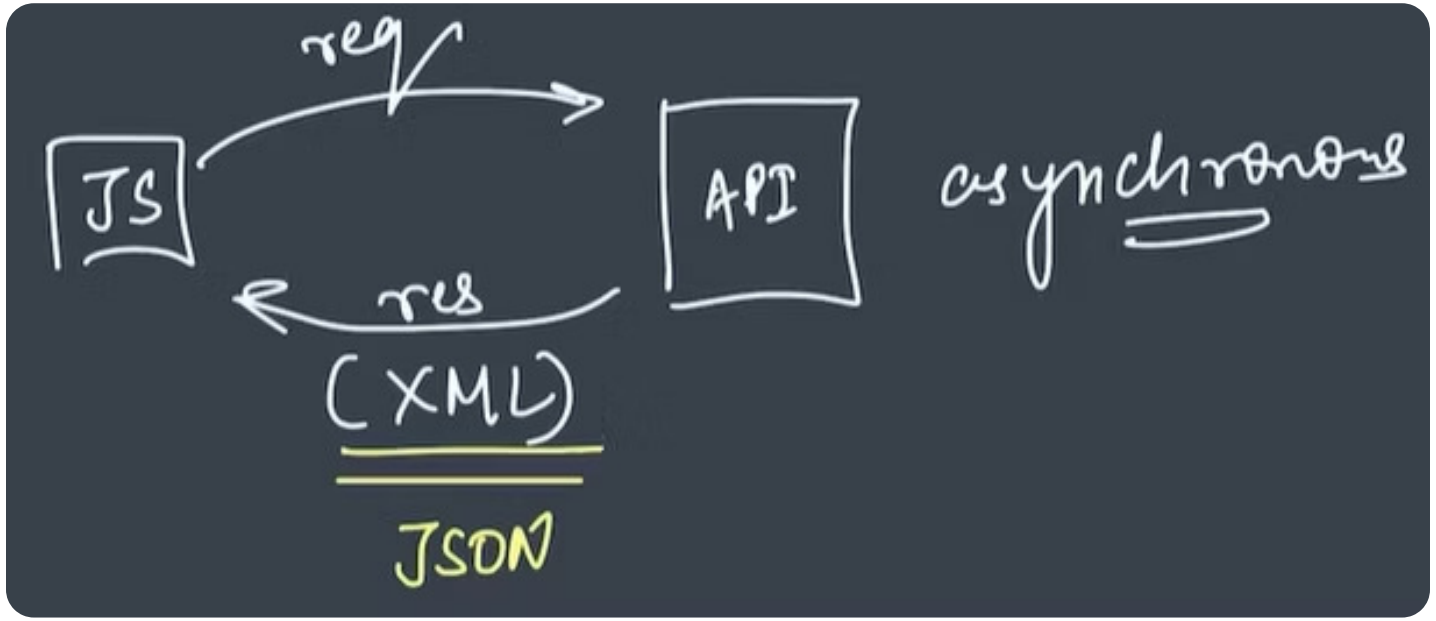
hoppscotch_io

Hoppscotch • Open source API development ecosystem

Helps you create requests faster, saving precious time on development.

AJAX

Asynchronous JavaScript and XML



HTTP Request and Status Code

Examples :

- GET
- POST
- DELETE

Examples :

- 200 - OK
- 404 - Not Found
- 400 - Bad Request
- 500 - Internal Server Error

Add Information in URLs

Query Strings

https://www.google.com/search?q=harry+porter

Key

Value

?name=shradha&marks=95

Our First Request

using **Fetch**

fetch(url)

```
let url = "https://catfact.ninja/fact";

fetch(url)
  .then((res) => {
    return res.json();
  })
  .then((data) => {
    console.log("data1 = ", data.fact);
    return fetch(url);
  })
  .then((res) => {
    return res.json();
  })
  .then((data2) => {
    console.log("data2 = ", data2.fact);
  })
  .catch((err) => {
    console.log("ERROR - ", err);
  });
```

```javascript
let url = "https://catfact.ninja/fact";

async function getFacts() {
  try {
    let res = await fetch(url);
    let data = await res.json();
    console.log(data.fact);

    let res2 = await fetch(url);
    let data2 = await res2.json();
    console.log(data2.fact);
  } catch (e) {
    console.log("error - ", e);
  }

  console.log("bye");
}
```

Axios

Library to make HTTP requests

```
async function getFacts() {
  try {
    let res = await axios.get(url);
    console.log(res);
  } catch (e) {
    console.log("ERROR - ", e);
  }
}
```